DV1703 Final Project

A League of Legends Match Statistics Database

Kevin Grundhall
kegr25@student.bth.se / kevin.grundhall01@gmail.com
https://github.com/krundhall/eggriot

Blekinge Institute of Technology

1. Project Idea

Playing League of Legends regularly with friends often gives rise to debates around player performance and in-game statistics. Who dealt the most damage, which champion wins the most, who makes the most efficient use of their gold? Rather than keeping these conversations hypothetical, this project aims to turn them into statistic-driven, data-backed discussions by building a database-backed application around real match data.

The application addresses the problem of scattered, hard-to-access match data by centralizing it into a structured, queryable database. The primary users are casual players who want to look up and compare their match history and performance statistics in a meaningful way.

To achieve this, I leveraged Riot Games' (the developer of League of Legends) publicly available match API to retrieve real player and match data. All data is sourced from their official developer platform at **https://developer.riotgames.com/apis** and **https://developer.riotgames.com/docs/lol**. This provides the project with a real-world data foundation, as opposed to relying on generated or synthetic data.

The core features of the application include looking up players by their in-game name, viewing match history, comparing champion performance statistics, and tracking item usage across games.

2. Schema Design

Image 1. E/R diagram representing project schema

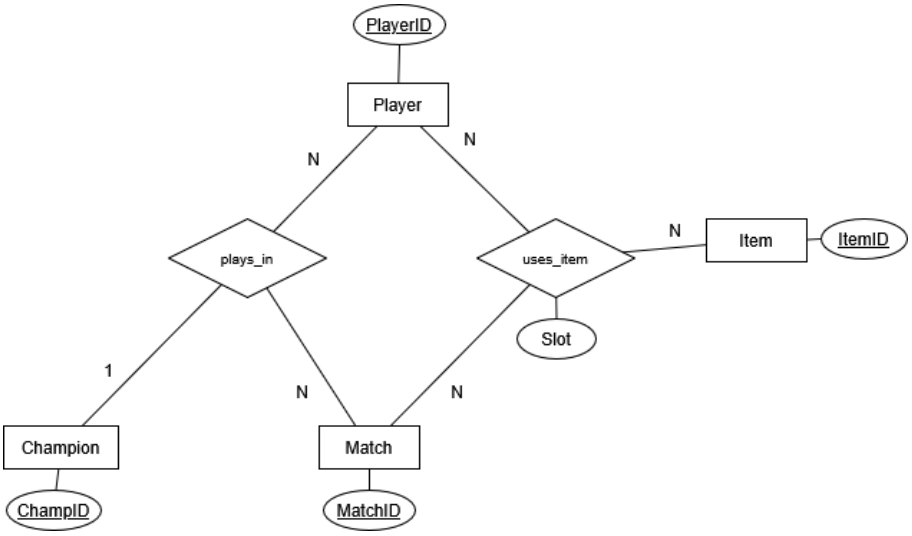
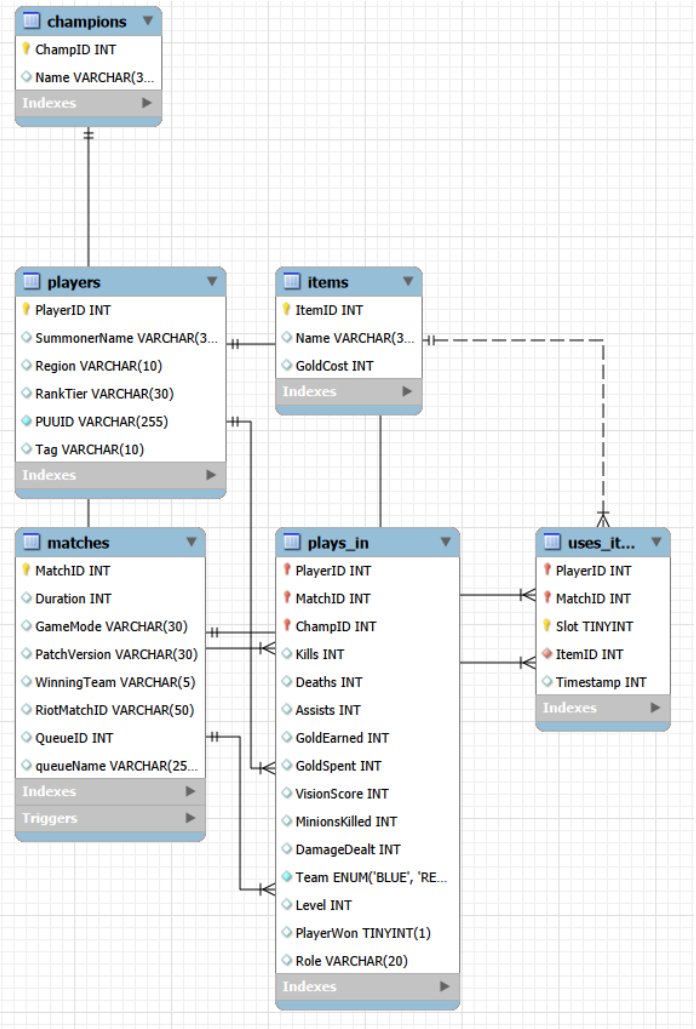


Image 2. EER diagram generated by mySQLworkbench showing all attributes



The data model is built around five entities: Players, Champions, Matches, Items and two relationship tables - plays_in and uses_items.

Players represent League of Legends accounts, identified by a unique ‘PlayerID’. A player carries attributes like ‘SummonerName’, ‘Region’, ‘RankTier’, ‘PUUID’ and ‘Tag’. *Champions* are the in-game characters a player can select, stored with a ‘ChampID’ and ‘Name’. *Matches* capture game-level metadata such as ‘Duration’, ‘GameMode’, ‘PatchVersion’, ‘WinningTeam’ and some queue information.

Incredibly pivotal to this project is the relationship ‘*plays_in’*. It is the center of the model. It connects a Player, a Match and the Champion that the player played, forming a composite primary key of (PlayerID, MatchID, ChampID). Since one player plays in many matches and one match involves many players, this is a many-many relationship. The relationship also stores stats from the game like ‘Kills’, ‘Deaths’, ‘Assists’, ‘GoldEarned’, ‘DamageDealt’ and ‘PlayerWon’. These might look like odd columns, but these are attributes that are representative of a player's participation in a match, not of the player or match in itself.

Items represent purchasable in-game weapons and/or equipment that's associated with a ‘GoldCost’. The ‘*uses_items*’ table resolves the many-many relationship between a players match and what items they bought during said match with ‘Slot’ and ‘Timestamp’ as additional attributes to track in what order they bought the items and when they bought the item.

I found that this design avoids redundancy. Champion and item data are stored once, and then referenced via foreign keys. This ensures data integrity across updates, and makes querying player and match statistics both efficient and logically straightforward.

3. SQL Queries

Q: **List the ten most likely items to be on the winning team and their win rate. The items must have occurred ten or more times to be statistically relevant.**
This query joins Items, uses_item and plays_in to calculate the win rate of each item. Items used in fewer than 10 matches are excluded to ensure statistical relevance. Results are sorted by win rate in descending order, returning the top 10.

```
SELECT i.Name, ROUND(AVG(PlayerWon) * 100, 2) AS WR
FROM items i
JOIN uses_item ui ON i.itemID = ui.ItemID
JOIN plays_in pi ON pi.PlayerID = ui.PlayerID AND pi.MatchID = ui.MatchID
GROUP BY i.Name
HAVING COUNT(*) >= 10
ORDER BY WR DESC
LIMIT 10
```

Q: Show the average KDA ratio for the top 10 best performers with a certain game tag.

This query joins plays_in and players to calculate average performance statistics per player. GREATEST(pi.Deaths, 1) is used in the KDA calculation to avoid division by zero for players with a perfect game. Players are filtered by summoner name using LIKE, and results are grouped per player, sorted by average KDA descending, returning the top 10.

```
SELECT p.SummonerName, p.Tag,
   ROUND(AVG(pi.Kills), 2) AS AvgKills,
   ROUND(AVG(pi.Deaths), 2) AS AvgDeaths,
   ROUND(AVG(pi.Assists), 2) AS AvgAssists,
   ROUND(AVG((pi.Kills + pi.Assists) / GREATEST(pi.Deaths, 1)), 2) AS AvgKDA
FROM plays_in pi
JOIN players p ON pi.PlayerID = p.PlayerID
WHERE p.SummonerName LIKE '% Egg'
GROUP BY pi.PlayerID, p.SummonerName, p.Tag
ORDER BY AvgKDA DESC
LIMIT 10
```

Q: Show the five longest matches played by people with a certain game tag.
This query joins plays_in, players and matches to retrieve match duration per player. Duration is converted from seconds to minutes using ROUND. Players are filtered by summoner name using LIKE, sorted by duration descending, returning the top 5.

```
SELECT p.SummonerName, p.Tag, m.RiotMatchID,
   ROUND(m.Duration / 60, 1) AS Duration_minutes
FROM plays_in pi
JOIN players p ON pi.PlayerID = p.PlayerID
JOIN matches m ON pi.MatchID = m.MatchID
WHERE p.SummonerName LIKE '% Egg'
ORDER BY m.Duration DESC
LIMIT 5
```

Q: Show the ten most purchased items, excluding trinkets (slot 6).
This query joins uses_item and items to count how many times each item has been purchased across all matches. Slot 6 is excluded as it represents trinkets, which are free utility items and not meaningful purchases. Results are grouped by item and sorted by purchase count descending, limiting to the top 10.

```
SELECT i.Name, COUNT(*) AS TimesBought
FROM uses_item ui
JOIN items i ON ui.ItemID = i.ItemID
WHERE ui.Slot != 6
GROUP BY ui.ItemID, i.Name
ORDER BY TimesBought DESC
LIMIT 10
```

Q: Show the ten most played champions across all matches.
This query joins plays_in and champions to count how many times each champion has been selected across all recorded matches. Results are grouped per champion and sorted by pick count descending, returning the top 10.

```
SELECT c.Name, COUNT(*) AS TimesPlayed
FROM plays_in pi
JOIN champions c ON pi.ChampID = c.ChampID
GROUP BY pi.ChampID, c.Name
ORDER BY TimesPlayed DESC
LIMIT 10
```

T: Automatically set the queue name based on queue ID on match insert.
This trigger fires before any insert on the matches table. It maps the QueueID to a readable queueName. 400 becomes "Normal SR", 420 becomes "Ranked SR", and any other value defaults to "Unknown". This avoids storing redundant name data in the source and keeps queue names consistent across all records automatically.

```
DELIMITER $$
CREATE TRIGGER t_insertQueueName
BEFORE INSERT ON matches
FOR EACH ROW BEGIN
   IF NEW.QueueID = 400 THEN
      SET NEW.queueName = "Normal SR";
   ELSEIF NEW.QueueID = 420 THEN
      SET NEW.queueName = "Ranked SR";
   ELSE
      SET NEW.queueName = "Unknown";
   END IF;
END $$
DELIMITER ;
```

P: Retrieve a full performance summary for a specific player by summoner name.
This procedure takes a summoner name as input and returns a complete stat summary — total matches, average kills, deaths, assists, KDA and win rate. It joins plays_in and players on PlayerID, filters by the provided name, and groups by player to aggregate all their match history into a single row.

```
DELIMITER $$
CREATE PROCEDURE GetPlayerSummary(IN p_name VARCHAR(30))
BEGIN
   SELECT p.SummonerName, p.Tag,
      COUNT(*) AS TotalMatches,
      ROUND(AVG(pi.Kills), 2) AS AvgKills,
      ROUND(AVG(pi.Deaths), 2) AS AvgDeaths,
      ROUND(AVG(pi.Assists), 2) AS AvgAssists,
      ROUND(AVG((pi.Kills + pi.Assists) / GREATEST(pi.Deaths, 1)), 2) AS AvgKDA,
      ROUND(AVG(pi.PlayerWon) * 100, 2) AS WinRate
   FROM plays_in pi
   JOIN players p ON pi.PlayerID = p.PlayerID
   WHERE p.SummonerName = p_name
   GROUP BY pi.PlayerID, p.SummonerName, p.Tag;
END$$
DELIMITER ;
```

4. Discussion and Resources

The main challenge during development was Riot Games API rate limits, which occasionally slowed down testing when fetching large amounts of match data. Outside of that, the API was clean and well documented, making data collection straightforward with no major inconsistencies to handle.
The Python side of the project was a highlight. Implementing the queries and building the application logic around real player data was more enjoyable then what I expected.

Source code: https://github.com/krundhall/eggriot
API: https://developer.riotgames.com/apis

Task	Date
Project setup, initial schema design	2026-02-07
E/R diagram drafted	2026-02-08
Riot API integration, simple match data fetching	2026-02-12
Core SQL queries	2026-02-15
Python match fetching connected to db	2026-02-21
Final cleanup, report written	2026-02-28
Final final cleanup of python program	2026-03-04

All work done by Kevin Grundhall (kegr25).